



**UNIVERSIDAD
DON BOSCO**

FACULTAD DE INGENIERÍA

ESCUELA DE COMPUTACIÓN

DESARROLLO DE SOFTWARE EMPRESARIAL

DESAFIO 1

Cloud Computing y Contenerización con Render + Docker

Sistema de Gestión de Películas

REPOSITORIO: <https://github.com/marcelavasquez11/MVCPeliculas-cloud.git>

DELMY MAJANO

INTEGRANTES:

MELISSA ABIGAIL FLORES ALFARO FA220709

WENDY MARCELA AGUILAR VASQUEZ AV220801

Desafío 1: Cloud Computing y Contenerización con Render + Docker

Aplicación seleccionada

Para el desarrollo del desafío se seleccionó la aplicación **MVCPelículas**, desarrollada utilizando **ASP.NET MVC**, **Entity Framework Core** y **SQL Server** como sistema gestor de base de datos.

Durante el desarrollo del proyecto se evaluó el despliegue en **Microsoft Azure**; sin embargo, debido a dificultades de configuración, se optó por utilizar **Render** para el despliegue de la aplicación y **Somee** como servicio de alojamiento gratuito para la base de datos **SQL Server**. Esta configuración permitió disponer de la aplicación en un entorno accesible mediante Internet y continuar con las actividades planteadas en el desafío.

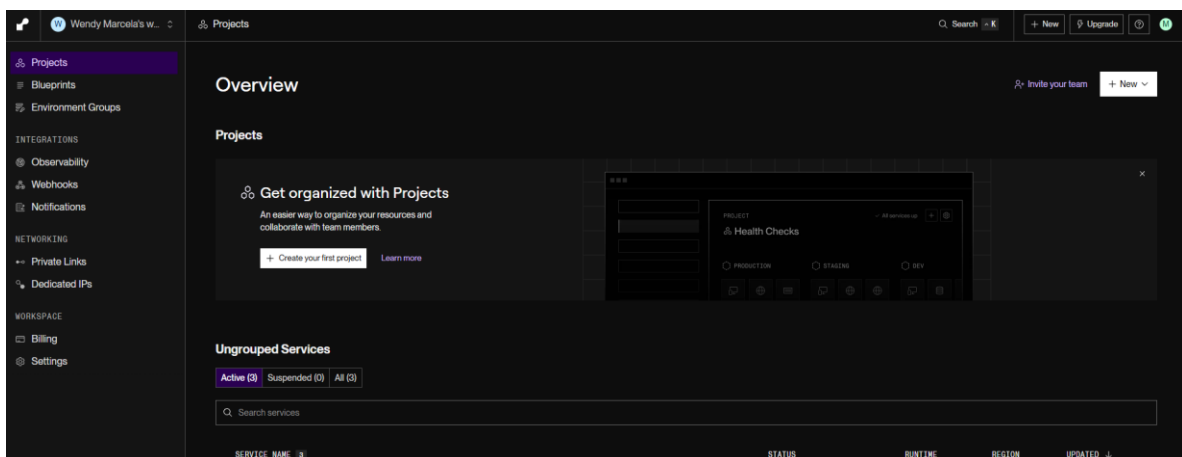
Desarrollo de los requerimientos funcionales

RF1. Creación de una cuenta en Render

Como plataforma de despliegue se utilizó **Render**. Para ello, se ingresó al sitio web oficial de Render y se creó una cuenta utilizando el correo institucional.

Procedimiento:

1. Ingresar al sitio web oficial de Render.
2. Seleccionar la opción **Sign Up**.
3. Registrarse utilizando el correo institucional.
4. Confirmar el registro y acceder al panel principal de Render.

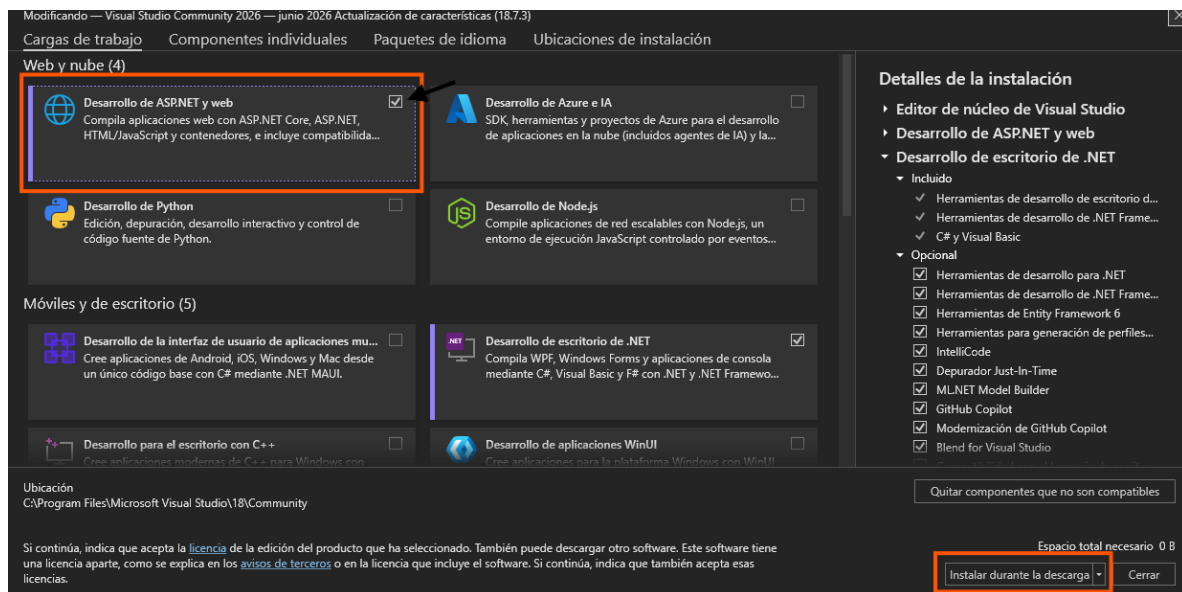


RF2. Instalación de las cargas de trabajo en Visual Studio

En este caso, únicamente fue necesario instalar la carga de trabajo Desarrollo de ASP.NET y web desde Visual Studio Installer, ya que esta proporciona las herramientas necesarias para el desarrollo y publicación de aplicaciones ASP.NET. Debido a que el despliegue se realizó mediante Render, no fue necesario instalar la carga de trabajo Desarrollo de Azure.

Procedimiento:

1. Abrir Visual Studio Installer.
2. Seleccionar la opción Modificar correspondiente a la versión de Visual Studio instalada.
3. En la sección Cargas de trabajo, marcar la opción Desarrollo de ASP.NET y web.
4. Hacer clic en el botón Instalar (o Modificar, si Visual Studio ya estaba instalado) y esperar a que finalice el proceso.



RF3. Publicación de aplicación en Render Web Service

Procedimiento:

1. Abrir el proyecto MVCPelículas en Visual Studio para comprobar que la aplicación funciona correctamente antes de iniciar el proceso de despliegue.
2. Con el propósito de facilitar el despliegue de la aplicación en Render, el proyecto será almacenado en un repositorio de GitHub, el cual será utilizado como origen del código fuente durante el proceso de publicación. Para ello, se ingresa a la

página oficial de GitHub y se crea un repositorio con el nombre Proyecto MVCPelículas.

3. Ingresar a la carpeta local del proyecto y, desde la raíz del proyecto, abrir una terminal CMD, PowerShell o Git Bash. Posteriormente, ejecutar los siguientes comandos:

```
git init
```

```
git add .
```

```
git commit -m "Proyecto MVCPelículas"
```

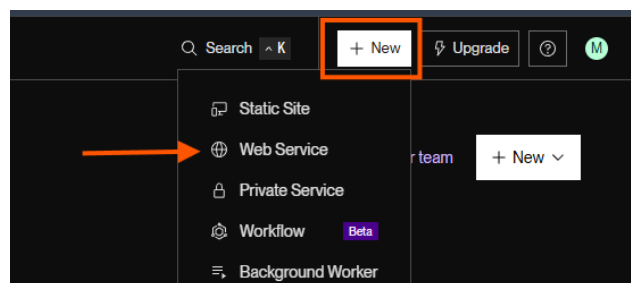
```
git remote add origin https://github.com/usuario/repositorio.git
```

```
git branch -M main
```

```
git push -u origin main
```

```
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\USUARIO\Desktop\MVCPelículas> git init
Initialized empty Git repository in C:/Users/USUARIO/Desktop/MVCPelículas/.git/
PS C:\Users\USUARIO\Desktop\MVCPelículas> git add .
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-grid.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-grid.min.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-grid.rtl.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-grid.rtl.min.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-reboot.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-reboot.min.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-reboot.rtl.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-reboot.rtl.min.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-utilities.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-utilities.min.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-utilities.rtl.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap-utilities.rtl.min.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap.min.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap.rtl.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/css/bootstrap.rtl.min.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/js/bootstrap.bundle.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/js/bootstrap.bundle.min.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/js/bootstrap.esm.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/js/bootstrap.esm.min.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/js/bootstrap.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/bootstrap/dist/js/bootstrap.min.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/jquery-validation-unobtrusive/dist/jquery.validate.unobtrusive.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/jquery-validation-unobtrusive/dist/jquery.validate.unobtrusive.min.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/jquery-validation/dist/jquery.validate.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/jquery-validation/dist/jquery.validate.min.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/jquery/dist/jquery.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/jquery/dist/jquery.min.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/jquery/dist/jquery.slim.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'MVCPelículas/wwwroot/lib/jquery/dist/jquery.slim.min.js', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\USUARIO\Desktop\MVCPelículas> git commit -m "Proyecto MVCPelículas"
[master (root-commit) c2d8a3b] Proyecto MVCPelículas
385 files changed, 92964 insertions(+)
total 1000000
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-grid.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-grid.min.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-grid.rtl.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-grid.rtl.min.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-reboot.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-reboot.min.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-reboot.rtl.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-reboot.rtl.min.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-utilities.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-utilities.min.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-utilities.rtl.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap-utilities.rtl.min.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap.min.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap.rtl.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\css\bootstrap.rtl.min.css
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\js\bootstrap.bundle.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\js\bootstrap.bundle.min.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\js\bootstrap.esm.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\js\bootstrap.esm.min.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\js\bootstrap.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\bootstrap\dist\js\bootstrap.min.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\jquery-validation-unobtrusive\dist\jquery.validate.unobtrusive.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\jquery-validation-unobtrusive\dist\jquery.validate.unobtrusive.min.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\jquery-validation\dist\jquery.validate.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\jquery-validation\dist\jquery.validate.min.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\jquery\dist\jquery.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\jquery\dist\jquery.min.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\jquery\dist\jquery.slim.js
-rw-r--r-- 1 Usuario Usuario 1000000 10/10/2023 10:10:10 AM MVCPelículas\wwwroot\lib\jquery\dist\jquery.slim.min.js
```

4. Una vez que el proyecto se encuentre almacenado en GitHub, iniciar sesión en Render con la cuenta institucional. En la página principal, seleccionar la opción New y, posteriormente, hacer clic en Web Service para crear un nuevo servicio.



5. En la siguiente pantalla, seleccionar la opción Public Git Repository, ingresar la URL del repositorio creado anteriormente y hacer clic en Connect.

New Web Service

Source Code

Git Provider Public Git Repository Existing Image

PR Previews and Auto-Deploy are available only for repositories configured with Blueprints

https://github.com/render-examples/express-hello-world

Connect

- Completar el formulario de configuración del servicio indicando los parámetros requeridos, como Name, Language, Region e Instance Type. Al finalizar, hacer clic en el botón Deploy Web Service para iniciar el proceso de publicación.

New Web Service

Source Code

marcelavasquez11 / MVCPelículas

Name

A unique name for your web service.

mvcpelículas-av220801

Project Optional

Add this web service to a project once it's created.

Create a new project to add this to?

You don't have any projects in this workspace. Projects allow you to group resources into environments so you can better manage related resources.

+ Create a project

Language

Choose the runtime environment for this service.

Docker

Branch

The Git branch to build and deploy.

main

Region

Your services in the same region can communicate over a private network. You can also connect to services in other regions.

Virginia (US East) 1 existing service

- Render iniciará la creación del servicio web y realizará el despliegue automático de la aplicación. Una vez finalizado el proceso, si el despliegue se realizó correctamente, el servicio mostrará el estado Live en color verde.

mvcpelículas-av220801-fa220709

WEB SERVICE

mvcpelículas-av220801-fa220709 Docker Free Upgrade your instance →

Service ID: srv-d9tme5dbedkc739jm6ng

marcelavasquez11 / MVCPelículas main f80f9ec ✓ Live

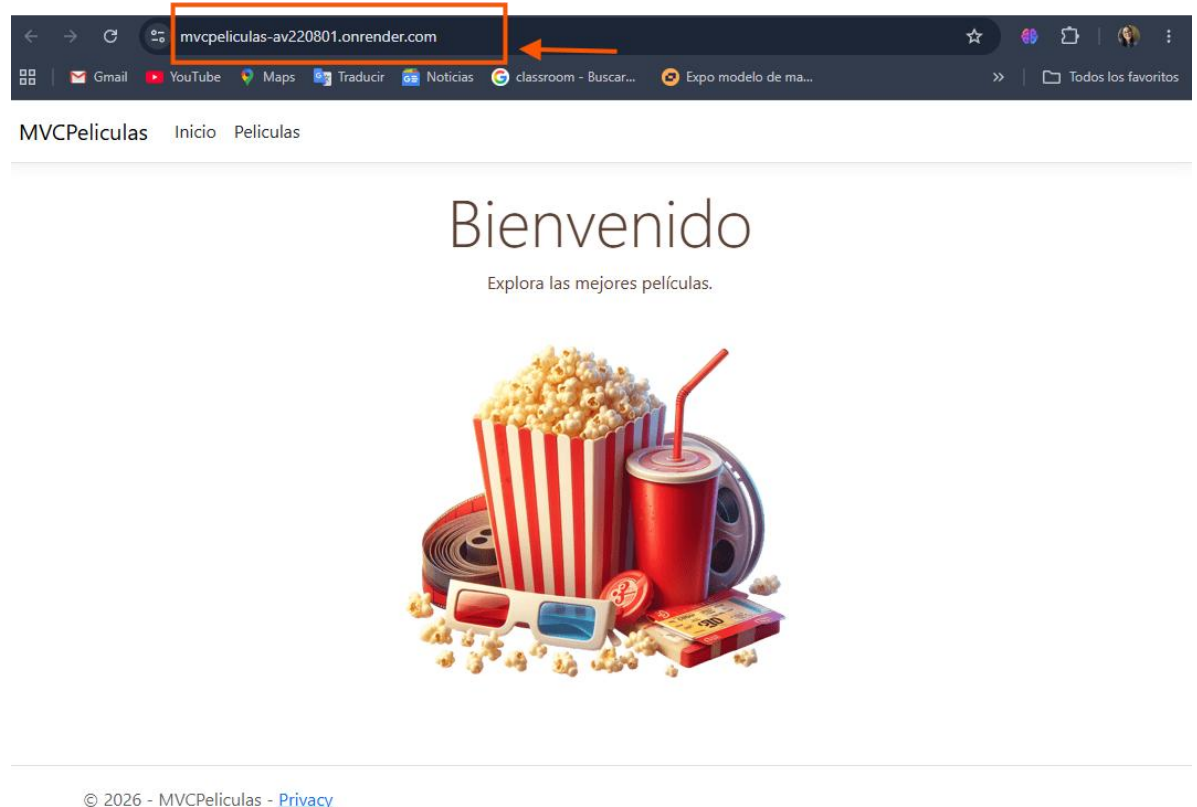
https://mvcpelículas-av220801.onrender.com

ⓘ Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

Search deploys and commits

- Finalmente, abrir la URL generada por Render para verificar que la aplicación se haya publicado correctamente y confirmar que la interfaz del sistema se visualiza y

funciona de manera adecuada.



RF4.Configuración de la base de datos SQL Server

Para alojar la base de datos utilizada por la aplicación, se empleó **Somee**, debido a que Render no ofrece alojamiento de bases de datos **SQL Server** en el entorno utilizado. Somee proporciona una opción gratuita para el alojamiento de bases de datos **SQL Server Express**, por lo que se utilizó este servicio como alternativa para mantener la base de datos accesible desde la aplicación desplegada en Render.

Procedimiento:

1. Ingresar a la página oficial de Somee y registrar una cuenta. Posteriormente, iniciar sesión en la plataforma.
2. En el dashboard principal, dirigirse al menú lateral izquierdo y seleccionar la opción Base de datos → Crear base de datos.

The screenshot shows the SOMEe.com dashboard. On the left, the 'Panel' menu is expanded, and the 'Bases de datos' (Databases) option is highlighted with a red box and an arrow. The main content area displays a welcome message 'Bienvenida, Wendy.' and a section titled 'Sus productos y servicios' (Your products and services). This section includes two cards: 'sitios web' (websites) and 'Bases de datos MS SQL' (MS SQL databases). The 'Bases de datos MS SQL' card shows '1 de 1' (1 of 1) and a 'Ver todo' (View all) link. Below the cards is a 'CREAR SITIO WEB' (Create website) button.

- Introducir el nombre Películas para la base de datos y seleccionar la opción Crear. Una vez creada, la plataforma mostrará la información correspondiente a la base de datos.

The screenshot shows the 'Bases de datos / Películas' (Databases / Movies) page. The page is divided into several sections. The top section, 'Detalles de la base de datos' (Database details), includes a list of actions: 'administrador de archivos de copia de seguridad' (backup file manager), 'Ejecutar scripts' (run scripts), 'Restaurar' (restore), 'Respaldo' (backup), 'Adjuntar' (attach), 'Obtener archivos de base de datos' (get database files), and 'Borrar' (delete). Below this, a card displays the database details:

- Nombre de la base de datos: Películas
- Nombre de dominio completo: Películas.mssql.somee.com
- Versión SQL: MS SQL 2022 Express
- Suscripción: Paquete de alojamiento gratuito (SPID1390624)
- ID del servicio: MPID5096383
- Facturas: INVID1507377

 A 'CAMBIAR SUSCRIPCIÓN' (change subscription) button is also present. To the right, a table shows the storage usage:

Archivo de datos	Capacidad: 30 MB
Usado: 8 MB	Gratis: 22 MB

 Another table shows the log file usage:

Archivo de registro	Capacidad: 30 MB
Usado: 8 MB	Gratis: 22 MB

 The bottom section, 'Detalles de conexión' (connection details), includes:

- Versión de SQL Server: MS SQL 2022 Express
- Dirección del servidor SQL: Películas.mssql.somee.com
- Nombre de usuario: marce_SQLLogin_1
- Contraseña de inicio de sesión: COPIAR AL PORTAPEPELES
- Cadena de conexión: COPIAR AL PORTAPEPELES

 A 'CAMBIAR CONTRASEÑA DE INICIO DE SESIÓN' (change login password) button is also present. The right side of the bottom section, 'Ubicación de copia de seguridad FTP' (FTP backup location), includes:

- DIRECCIÓN: ftp://Películas.backup.somee.com/Películas_MSSql_Database_Backup
- Nombre de usuario: marce
- Contraseña: * Utilice la contraseña de este panel de control *

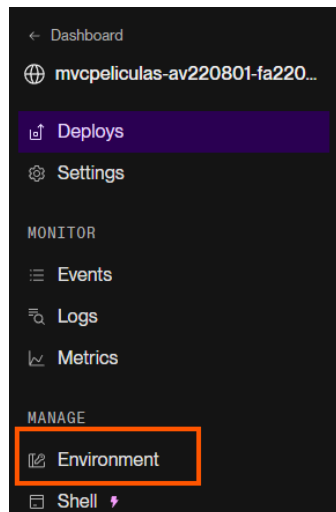
 An 'ADMINISTRADOR DE ARCHIVOS DE COPIA DE SEGURIDAD' (backup file manager) button is also present.

RF5: Conexión aplicación ↔ Somee SQL

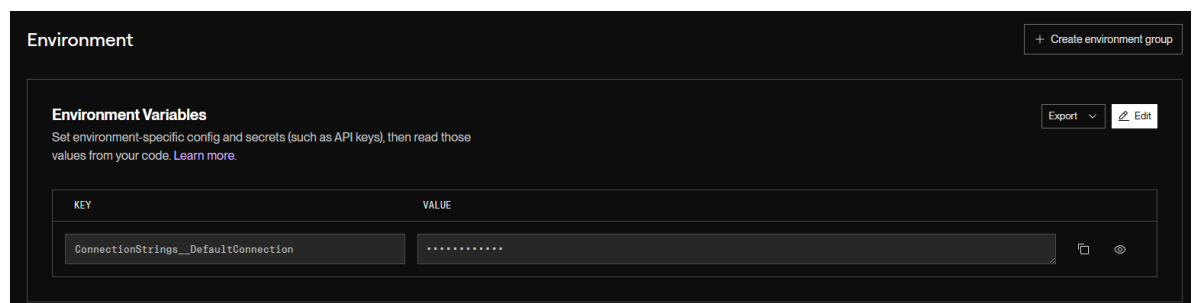
Para conectar la aplicación desplegada en Render con la base de datos SQL Server alojada en Somee, se configuró la cadena de conexión mediante una variable de entorno.

Procedimiento:

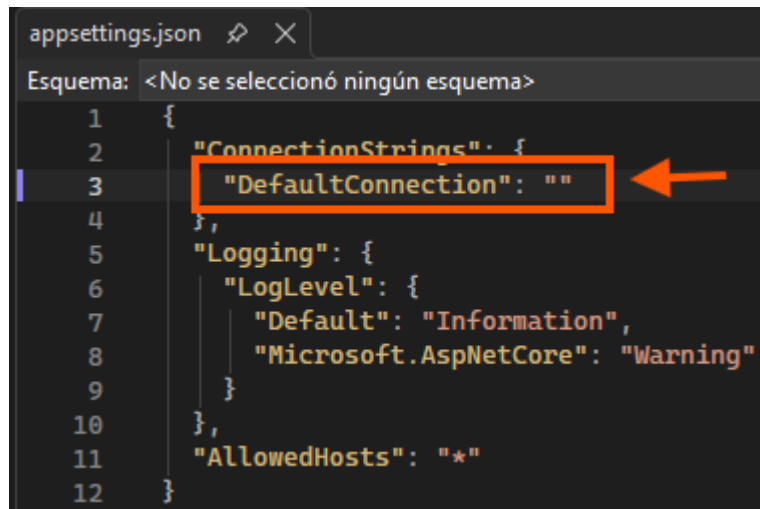
1. Ingresar nuevamente a Render y acceder al Web Service correspondiente al proyecto.
2. En el menú lateral izquierdo, seleccionar la opción **Environment**.



3. Creamos una nueva variable de entorno con el nombre `ConnectionStrings__DefaultConnection` y en el campo Value, introducir la cadena de conexión proporcionada por Somee.

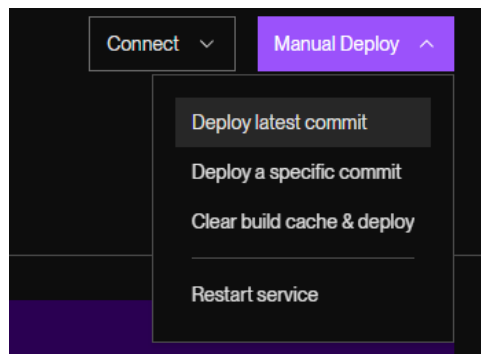


4. Modificar la configuración del proyecto. Debido a que anteriormente la aplicación utilizaba una conexión local a SQL Server, se eliminó la cadena de conexión almacenada directamente en `appsettings.json`, dejando la configuración preparada para obtener el valor mediante la variable de entorno.



```
appsettings.json
Esquema: <No se seleccionó ningún esquema>
1  {
2    "ConnectionStrings": {
3      "DefaultConnection": ""
4    },
5    "Logging": {
6      "LogLevel": {
7        "Default": "Information",
8        "Microsoft.AspNetCore": "Warning"
9      }
10   },
11   "AllowedHosts": "*"
12 }
```

5. Ejecutar las migraciones de Entity Framework Core para actualizar la estructura de la base de datos.
6. Realizar las pruebas correspondientes para comprobar que la aplicación puede acceder correctamente a la base de datos alojada en Somee.
7. Subir los cambios realizados al repositorio de GitHub.
8. En Render, realizar un nuevo Deploy utilizando el último commit del repositorio.



9. Finalmente, verificar que la aplicación desplegada en Render pueda consultar y modificar correctamente la información almacenada en la base de datos de Somee.

Introduzca su consulta SQL o lote T-SQL. Se ejecutará en la base de datos: Peliculas

```
SELECT * FROM Peliculas
```

EJECUTAR T-SQL

Conjunto 1. Total de filas: 6

Identificación	Título	FechaLanzamiento	Precio	Director	Generoid
3	Harry Potter y la piedra filosofal	16/11/2001 00:00:00	7.5500	Chris Columbus	1
4	El Señor de los Anillos: La Comunidad del Anillo	12/10/2001 00:00:00	8.3000	Peter Jackson	3
5	El silencio de los corderos	14/2/1991 00:00:00	6.2500	Jonathan Demme	2
6	Shrek	13/08/2026 00:00:00	7.5500	Chris Columbus	4
7	Amnesia	13/08/2026 00:00:00	9.5000	Peter Jackson	5
9	Moana 2	8/7/2026 12:00:00 AM	3.0000	Melis	1

Introduzca su consulta SQL o lote T-SQL. Se ejecutará en la base de datos: Peliculas

```
SELECT * FROM Generos
```

EJECUTAR T-SQL

Conjunto 1. Total de filas: 5

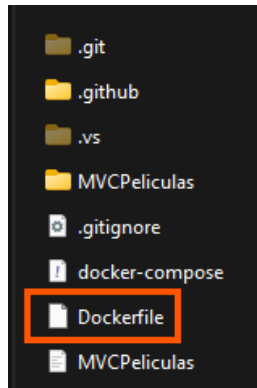
Identificación	Nombre
1	Fantasia
2	Drama
3	Aventura
4	Comedia
5	Terror

RF6. Contenerización con Docker

En esta sección se realiza el proceso de contenerización de la aplicación mediante Docker. Para ello, se configura el entorno necesario y se crea una imagen Docker que incluye la aplicación y todas sus dependencias, permitiendo su ejecución de forma consistente en diferentes entornos sin necesidad de realizar configuraciones adicionales.

Procedimiento :

1. Abrir la carpeta del proyecto MVCPeliculas en la raíz
2. Crear un archivo denominado Dockerfile.



3. Configurar el archivo **Dockerfile** con las instrucciones necesarias para compilar, publicar y ejecutar la aplicación utilizando una imagen de .NET 8.
4. Abrir una terminal en la carpeta raíz del proyecto y ejecutar el siguiente comando para construir la imagen Docker:

`docker build -t mvcpeliculas .`

```
PS C:\Users\USUARIO\Desktop\MVCPeliculas> docker build -t mvcpeliculas .
[+] Building 71.0s (18/18) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 764B                                0.0s
=> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:8.0-alpine 1.4s
=> [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:8.0-alpine 1.4s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [build 1/8] FROM mcr.microsoft.com/dotnet/sdk:8.0-alpine@sha256:8a80a27ddac789b4cb6d09d244 26.7s
=> => resolve mcr.microsoft.com/dotnet/sdk:8.0-alpine@sha256:8a80a27ddac789b4cb6d09d244f9c8d84 0.0s
=> sha256:55d326ceb2d4e228242a281f553f65795909072f42abd198f3447b139973bb4 17.60MB / 17.60MB 3.9s
=> sha256:42408e2b3972cc7bf4f211d814c60c9681b6a37d715bd0d5813c66ba6aa58c68 2.64kB / 2.64kB 3.8s
=> sha256:6df907eac5a5056532bf93ac167302a7819bee09e7280efe9a8844bcbac4 177.90MB / 177.90MB 21.7s
=> sha256:adb29788ec17156b1e7afbeff69fa41406568c041d5d6b881244a4205239c7 22.19MB / 22.19MB 14.5s
=> extracting sha256:adb29788ec17156b1e7afbeff69fa41406568c041d5d6b881244a4205239c7 0.4s
=> extracting sha256:6df907eac5a5056532bf93ac167302a7819bee09e7280efe9a8844bcbac4 2.4s
=> extracting sha256:42408e2b3972cc7bf4f211d814c60c9681b6a37d715bd0d5813c66ba6aa58c68 0.0s
=> extracting sha256:55d326ceb2d4e228242a281f553f65795909072f42abd198f3447b139973bb4 0.3s
=> [final 1/3] FROM mcr.microsoft.com/dotnet/aspnet:8.0-alpine@sha256:b288317d8ed45bb763fa95db 8.4s
=> => resolve mcr.microsoft.com/dotnet/aspnet:8.0-alpine@sha256:b288317d8ed45bb763fa95db807cf 0.0s
=> sha256:f94543cc58b8de85d8dca45d67218802929ffbdee3e2cbf392f367c9d47ee3b 11.10MB / 11.10MB 1.6s
=> sha256:24cd951222aed29d627837cba36d6a4b51515df2ca3d06ed13fc4ab148860843 164B / 164B 0.6s
=> sha256:16a998e823e1a1a106278de16647a2778c1171706a498ca6526fee973008f48 32.27MB / 32.27MB 7.6s
=> sha256:73368dcd9fe86594c6ddf02fc920ec99ca93e23d4930855fc1fd82f3f3142f0d 945B / 945B 0.7s
=> sha256:2a1956658526fd51902fbc5b1fb0b996f4515fa5b2a9f8f11b7bc00df5daa6fe 1.09MB / 1.09MB 1.1s
=> sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4 3.85MB / 3.85MB 1.4s
=> extracting sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4 0.1s
=> extracting sha256:2a1956658526fd51902fbc5b1fb0b996f4515fa5b2a9f8f11b7bc00df5daa6fe 0.0s
=> extracting sha256:73368dcd9fe86594c6ddf02fc920ec99ca93e23d4930855fc1fd82f3f3142f0d 0.0s
=> extracting sha256:16a998e823e1a1a106278de16647a2778c1171706a498ca6526fee973008f48d 0.4s
=> extracting sha256:24cd951222aed29d627837cba36d6a4b51515df2ca3d06ed13fc4ab148860843 0.0s
=> extracting sha256:f94543cc58b8de85d8dca45d67218802929ffbdee3e2cbf392f367c9d47ee3b0 0.1s
=> [internal] load build context                                    0.4s
=> => transferring context: 3.63MB                                    0.3s
=> [final 2/3] WORKDIR /app                                        0.4s
=> [build 2/8] WORKDIR /src                                        2.5s
=> [build 3/8] COPY [MVCPeliculas/MVCPeliculas.csproj, MVCPeliculas/] 0.1s
=> [build 4/8] RUN dotnet restore "MVCPeliculas/MVCPeliculas.csproj" 22.6s
=> [build 5/8] RUN dotnet tool install --global dotnet-ef --version 8.* 3.0s
=> [build 6/8] COPY . .                                           2.5s
=> [build 7/8] WORKDIR /src/MVCPeliculas                          0.1s
=> [build 8/8] RUN dotnet build "MVCPeliculas.csproj" -c Release -o /app/build 5.7s
=> [publish 1/1] RUN dotnet publish "MVCPeliculas.csproj" -c Release -o /app/publish /p:UseApp 3.4s
=> [final 3/3] COPY --from=publish /app/publish .                 0.7s
```

5. Una vez finalizada la construcción de la imagen, ejecutar un contenedor utilizando el siguiente comando:

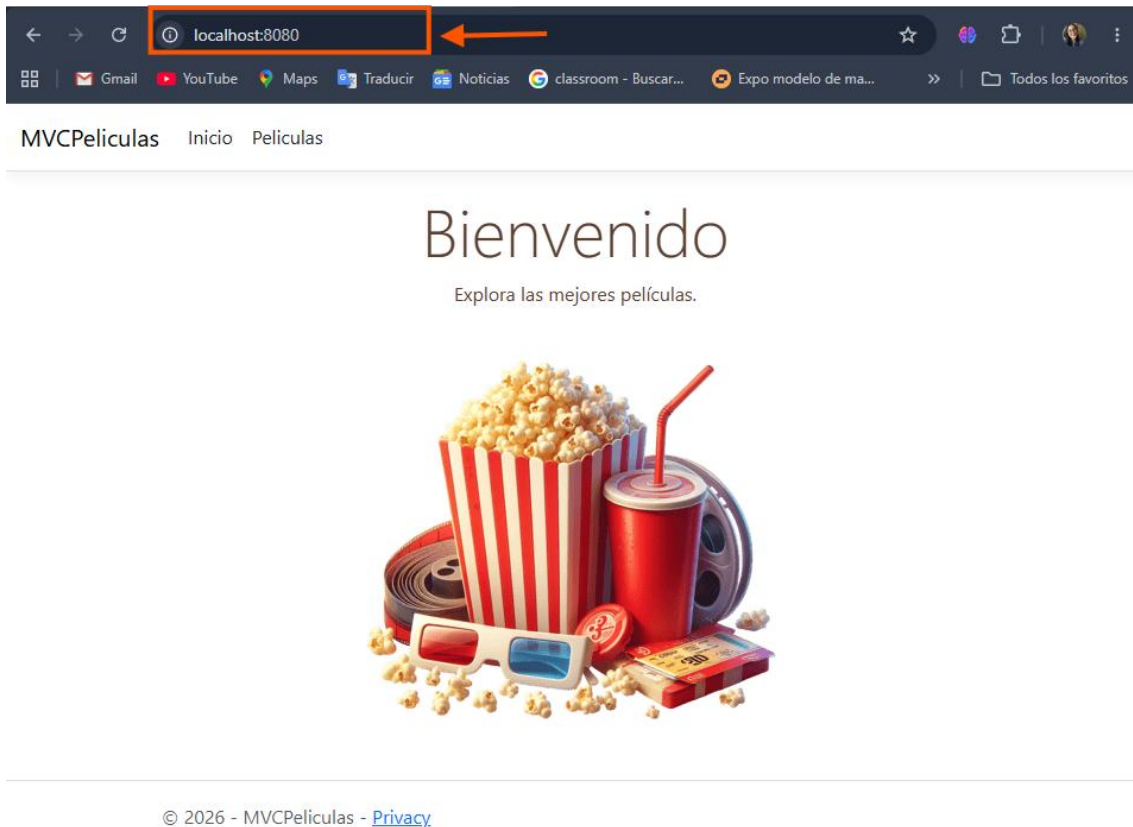
`docker run -p 8080:8080 mvcpeliculas`

```
PS C:\Users\USUARIO\Desktop\MVCPeliculas> docker run --rm -p 8080:8080 -e "ConnectionStrings__DefaultConnection=workstation id=Películas.mssql.somee.com;packet size=4096;user id=marce_SQLLogin_1;pwd=cbdvvkkk7fj;data source=Películas.mssql.somee.com;persist security info=False;initial catalog=Películas;TrustServerCertificate=True" mvcpeliculas
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (82ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 1
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (80ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT OBJECT_ID(N'[__EFMigrationsHistory]');
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (73ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 1
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (76ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT OBJECT_ID(N'[__EFMigrationsHistory]');
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (100ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [MigrationId], [ProductVersion]
      FROM [__EFMigrationsHistory]
      ORDER BY [MigrationId];
info: Microsoft.EntityFrameworkCore.Migrations[20405]
      No migrations were applied. The database is already up to date.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
```

6. Abrir un navegador web e ingresar a:

<http://localhost:8080>

para verificar que la aplicación se ejecuta correctamente desde el contenedor.



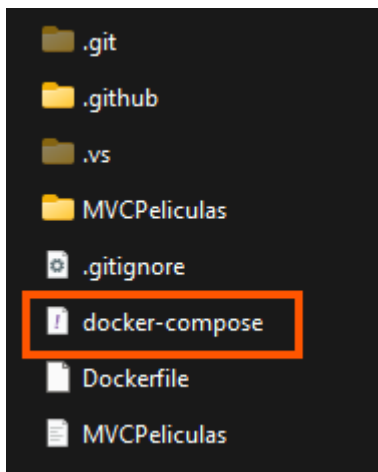
RF7. Orquestación multi-contenedor con Docker Compose

Para complementar la contenerización de la aplicación, se configuró Docker Compose con el propósito de ejecutar de manera conjunta la aplicación MVCPeliculas y el servidor de base de datos SQL Server en contenedores independientes.

La configuración permite que ambos contenedores se comuniquen mediante la red creada automáticamente por Docker Compose, permitiendo que la aplicación acceda a la base de datos utilizando el nombre del servicio definido en el archivo `docker-compose.yml`.

Procedimiento:

1. Crear el archivo `docker-compose.yml` en la carpeta raíz del proyecto MVCPeliculas.



2. Configurar el archivo para definir los servicios correspondientes a la aplicación y a la base de datos SQL Server.

```
1  services:
2
3  # Contenedor de la aplicación ASP.NET MVC
4  peliculas-app:
5    build: .
6    ports:
7      - "8081:8080"
8    depends_on:
9      - peliculas-db
10   # Cadena de conexión hacia el contenedor de SQL Server
11   environment:
12     - ConnectionStrings__DefaultConnection=Server=peliculas-db;Database=Películas;User Id=sa;Password=Your_password123!;TrustServerCertificate=True
13
14
15  # Contenedor de SQL Server
16  peliculas-db:
17    image: mcr.microsoft.com/mssql/server:2022-latest
18    environment:
19      - ACCEPT_EULA=Y
20      - MSSQL_SA_PASSWORD=Your_password123!
21    ports:
22      - "1433:1433"
23    # Mantiene los datos de SQL Server aunque el contenedor se elimine
24    volumes:
25      - sql-data:/var/opt/mssql
26
27
28  # Volumen utilizado para almacenar los datos de la base de datos
29  volumes:
30    sql-data:
```

3. Abrir una terminal en la carpeta raíz del proyecto y ejecutar:

docker compose up -d

Este comando permite crear e iniciar los contenedores definidos en el archivo docker-compose.yml.

4. Abrir **Docker Desktop** y verificar que los dos contenedores se encuentren ejecutándose correctamente con el comando docker ps.

```
PS C:\Users\USUARIO\Desktop\MVCPeliculas> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
f1f15c9e90d9	mvcpelículas-películas-app	"dotnet MVCPelículas..."	About a minute ago	Up 49 seconds
0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp				
55c059595b38	mcr.microsoft.com/mssql/server:2022-latest	"/opt/mssql/bin/Laun..."	43 hours ago	Up About a min
ute 0.0.0.0:1433->1433/tcp, [::]:1433->1433/tcp				

5. Acceder a la aplicación mediante el navegador y comprobar que funciona correctamente.
6. Verificar que los datos mostrados por la aplicación provienen de la base de datos SQL Server ejecutándose dentro del contenedor.

localhost:8081/Películas

MVCPelículas Inicio Películas

Index

[Crear Nuevo](#)

Título: [Buscar](#)

Título	Fecha de lanzamiento	Precio	Director	Género
Harry Potter y la piedra filosofal	11/16/2001	7.55	Chris Columbus	Fantasia
El Señor de los Anillos: La Comunidad del Anillo	12/10/2001	8.30	Peter Jackson	Aventura
El silencio de los corderos	02/14/1991	6.25	Jonathan Demme	Drama
Amnesia	08/14/2026	9.50	Christopher	Terror

[Editar](#) | [Detalles](#) | [Eliminar](#)

[Editar](#) | [Detalles](#) | [Eliminar](#)

[Editar](#) | [Detalles](#) | [Eliminar](#)

[Editar](#) | [Detalles](#) | [Eliminar](#)

Pruebas de funcionamiento

Con el propósito de verificar el correcto funcionamiento de la aplicación después de su despliegue y contenerización, se realizaron pruebas de acceso y ejecución utilizando los diferentes entornos configurados.

➤ URL pública de la aplicación

La aplicación MVCPeliculas se encuentra desplegada en Render, desde donde puede ser accedida mediante una URL pública.

URL: <https://mvcpeliculas-av220801.onrender.com>

➤ Comandos utilizados en Docker

Durante la contenerización y orquestación de la aplicación se utilizaron los siguientes comandos:

```
# Construir la imagen Docker
docker build -t mvcpeliculas .

# Ejecutar la aplicación en un contenedor
docker run -p 8080:8080 mvcpeliculas

# Construir y ejecutar los servicios con Docker Compose
docker compose up -d

# Verificar los contenedores en ejecución
docker ps

# Detener los servicios de Docker Compose
docker compose down
```

Conclusión

Durante el desarrollo del desafío, se realizó el proceso de despliegue, configuración y contenerización de la aplicación MVCPeliculas. Esto permitió ver cómo funcionaba la aplicación en un entorno al que se podía acceder mediante Internet. Primero, la aplicación se desplegó en Render. También se configuró una base de datos SQL Server con Somee.

Luego, se utilizó Docker para contenerizar la aplicación. Para ello, se creó una imagen a partir de un Dockerfile. Finalmente, se configuró Docker Compose para que la aplicación y el servidor de base de datos se ejecuten juntos en contenedores independientes. Esto facilita su comunicación y administración.

Con estas actividades, se entendió de manera práctica cómo se usan los servicios en la nube y las tecnologías de contenerización. También se vio la importancia de utilizar entornos reproducibles para facilitar el despliegue y la ejecución de aplicaciones.